

FTC Programming 101

Everything a new FTC programmer needs to go from zero to driving the robot — the tools, the Java, and your first OpModes. Pairs with the **STEM4U FTC Control Hub Setup** guide.

1 The big picture — how an FTC robot runs

Three things work together:

- **Control Hub** — the small computer bolted to the robot. It runs the **Robot Controller app** and drives the motors/servos.
- **Driver Station** — a phone/tablet the drivers hold. It has the gamepads and the big **Init / Play / Stop** buttons.
- **Your code** — small programs called **OpModes** that you write, then push onto the Control Hub.

So the job is: write an OpMode → push it to the Control Hub → pick it on the Driver Station → press Play.

OpMode = "Operational Mode" — one program for the robot. There are two kinds:

- **TeleOp** — driver-controlled (you move the robot with the gamepads).
- **Autonomous** — the robot runs by itself with no driver, usually the first 30 seconds of a match.

2 Your toolkit

- **Android Studio** — the program (an **IDE**) where you write Java and push code to the hub. (Simpler options exist: **Blocks** = drag-and-drop in a browser, and **OnBot Java** = type Java in a browser. Teams graduate to Android Studio for real projects.)
- **The FTC SDK** — FIRST's starter project, `FtcRobotController`. Your OpModes go in the `TeamCode` folder. It already knows how to talk to motors, sensors, and gamepads.
- **git & GitHub** — see §6. Short version: **git** saves snapshots of your code and lets a whole team work on it without overwriting each other; **GitHub** is the website that stores the team's shared copy.

3 The Java you actually need

FTC is written in **Java**. You don't need all of Java — just these building blocks:

- **Variables & types** — a labeled box that holds a value: `int` (whole number), `double` (decimal), `boolean` (true/false), `String` (text).
- **if / else** — make a decision based on whether something is true.
- **while / for** — repeat code (a loop).
- **Methods** — a named, reusable block of code you can "call."
- **Classes** — a container for your code. **Every OpMode is a class.** Don't worry about the deep theory yet — follow the template.

```

int    motorCount = 4;        // a whole number (integer)
double power      = 0.75;     // a decimal number
boolean armUp     = false;    // true or false
String teamName   = "WolfBots"; // text

// IF / ELSE – make a decision
if (gamepad1.a) {
    armUp = true;              // runs only when the A button is pressed
} else {
    armUp = false;
}

// WHILE – repeat until something changes
while (opModeIsActive()) {
    // this runs over and over until the match stops
}

// FOR – repeat a set number of times
for (int i = 0; i < 4; i++) {
    // runs 4 times (i = 0,1,2,3)
}

// A METHOD – a named, reusable block of code
double half(double x) {
    return x * 0.5;
}

```

Java rules to remember: every statement ends with a semicolon `;` · blocks are wrapped in curly braces `{ }` · `=` means "put into" while `==` means "is equal to" · Java cares about Capital Letters (`leftDrive` $\neq$ `LeftDrive`).

4 Anatomy of an OpMode

Every OpMode follows the same shape. Learn this once and the rest is filling in the blanks:

- **Annotation** — `@TeleOp` or `@Autonomous` tells the Driver Station to list your program (and under what name).
- **extends LinearOpMode** — the easiest OpMode style: your code runs top-to-bottom inside `runOpMode()`.
- **hardwareMap** — how your code grabs a motor/servo by the **name set in the robot's configuration**. The names must match exactly.
- **waitForStart()** — pauses until the driver presses Play.
- **while (opModelsActive())** — the main loop that runs during the match.
- **telemetry** — prints live info to the Driver Station screen (your #1 debugging tool).

The robot configuration (do this first!): on the Driver Station you build a **configuration** that lists every motor and servo, which port it's plugged into, and its **name**. The name you type there (e.g. `frontLeft`) must match the name in `hardwareMap.get(...)` **exactly**. Mismatched names are the #1 reason a new OpMode crashes.

5 Your first TeleOp — drive a mecanum robot

Mecanum wheels let the robot do something tank drive can't: **strafe sideways**. The left stick drives (forward/back + sideways), the right stick turns. Four motors, mixed together:

```
@TeleOp(name = "Mecanum Drive", group = "Teaching")
public class MecanumDrive extends LinearOpMode {

    @Override
    public void runOpMode() {
        // 1) Grab all FOUR motors (names must match the robot configuration!)
        DcMotor frontLeft = hardwareMap.get(DcMotor.class, "frontLeft");
        DcMotor frontRight = hardwareMap.get(DcMotor.class, "frontRight");
        DcMotor backLeft = hardwareMap.get(DcMotor.class, "backLeft");
        DcMotor backRight = hardwareMap.get(DcMotor.class, "backRight");

        // The right-side motors are mirrored
        frontRight.setDirection(DcMotor.Direction.REVERSE);
        backRight.setDirection(DcMotor.Direction.REVERSE);

        telemetry.addData("Status", "Ready - press PLAY");
        telemetry.update();
        waitForStart();

        // 2) The main loop - runs until the match ends
        while (opModeIsActive()) {
            double drive = -gamepad1.left_stick_y;    // forward / back
            double strafe = gamepad1.left_stick_x;    // sideways (mecanum!)
            double turn = gamepad1.right_stick_x;    // rotate

            // Mix the three inputs into four wheel powers
            double fl = drive + strafe + turn;
            double fr = drive - strafe - turn;
            double bl = drive - strafe + turn;
            double br = drive + strafe - turn;

            // Scale them together so none exceeds 1.0
            double max = Math.max(1.0, Math.max(Math.abs(fl), Math.max(Math.abs(fr),
                Math.max(Math.abs(bl), Math.abs(br)))));

            frontLeft.setPower(fl / max);
            frontRight.setPower(fr / max);
            backLeft.setPower(bl / max);
            backRight.setPower(br / max);

            telemetry.addData("drive/strafe/turn", "%.2f / %.2f / %.2f", drive, strafe, turn);
            telemetry.update();
        }
    }
}
```

Two things to know: (1) on a gamepad, pushing a stick **up is negative**, so we flip it with `-` to make "up" = forward. (2) We divide every wheel power by `max` so if the math pushes a wheel past 1.0, all four scale down together and the robot still drives straight. If a wheel spins the wrong way, flip that motor's `setDirection`.

6 Robot-centric vs field-centric driving

There are two ways to read the driver's sticks:

- **Robot-centric** (what §5 does) — "forward" means the **robot's** front. Push up and it goes the way *it* is facing. Simple and reliable, but the driver must think from the robot's point of view — confusing when the robot faces back at them.
- **Field-centric** — "forward" always means the **same direction on the field** (away from the driver), no matter which way the robot is turned. Push up and it moves away from you even if the robot is sideways. Much easier to drive; needs the **IMU** (the orientation sensor built into the Control Hub).

Field-centric works by reading the robot's **heading** from the IMU and rotating the joystick direction by that angle before powering the wheels:

```
import com.qualcomm.robotcore.hardware.IMU;
import org.firstinspires.ftc.robotcore.external.navigation.AngleUnit;

// --- set up the IMU once (tell it how the hub is mounted) ---
IMU imu = hardwareMap.get(IMU.class, "imu");
imu.initialize(new IMU.Parameters(new RevHub0OrientationOnRobot(
    RevHub0OrientationOnRobot.LogoFacingDirection.UP,
    RevHub0OrientationOnRobot.UsbFacingDirection.FORWARD)));

while (opModeIsActive()) {
    double y = -gamepad1.left_stick_y; // forward
    double x = gamepad1.left_stick_x; // strafe
    double rx = gamepad1.right_stick_x; // turn

    if (gamepad1.options) imu.resetYaw(); // re-zero "forward"

    // which way is the robot pointing right now?
    double heading = imu.getRobotYawPitchRollAngles().getYaw(AngleUnit.RADIANS);

    // rotate the driver's stick direction by -heading -> field-centric
    double rotX = x * Math.cos(-heading) - y * Math.sin(-heading);
    double rotY = x * Math.sin(-heading) + y * Math.cos(-heading);

    double max = Math.max(1.0, Math.abs(rotY) + Math.abs(rotX) + Math.abs(rx));
    frontLeft.setPower( (rotY + rotX + rx) / max);
    backLeft.setPower( (rotY - rotX + rx) / max);
    frontRight.setPower((rotY - rotX - rx) / max);
    backRight.setPower( (rotY + rotX - rx) / max);
}
```

Re-zero before the match. "Forward" is whichever way the robot faced when the IMU started (or when you last reset it), so have the driver press a button to `resetYaw()` while the robot points downfield. **Which to teach first?** Start **robot-centric** so students understand the wheel mixing, then upgrade the same OpMode to **field-centric**.

7 Your first Autonomous — move by itself

The simplest autonomous: drive forward for one second, then stop. (Later you'll use **encoders** — counters inside the motors — to drive an exact distance instead of a time.)

```
@Autonomous(name = "Drive Forward 1s", group = "Teaching")
public class DriveForward extends LinearOpMode {

    @Override
    public void runOpMode() {
        DcMotor frontLeft = hardwareMap.get(DcMotor.class, "frontLeft");
        DcMotor frontRight = hardwareMap.get(DcMotor.class, "frontRight");
        DcMotor backLeft = hardwareMap.get(DcMotor.class, "backLeft");
        DcMotor backRight = hardwareMap.get(DcMotor.class, "backRight");
        frontRight.setDirection(DcMotor.Direction.REVERSE);
        backRight.setDirection(DcMotor.Direction.REVERSE);

        waitForStart();

        setAll(frontLeft, frontRight, backLeft, backRight, 0.5); // all forward
        sleep(1000); // for 1 second
        setAll(frontLeft, frontRight, backLeft, backRight, 0.0); // stop
    }

    // a helper METHOD so we don't repeat ourselves
    void setAll(DcMotor a, DcMotor b, DcMotor c, DcMotor d, double p) {
        a.setPower(p); b.setPower(p); c.setPower(p); d.setPower(p);
    }
}
```

8 Patterns & gotchas every team uses

Hide an OpMode with @Disabled

As a team writes more programs, the Driver Station list gets crowded. Put `@Disabled` above a class to **keep the code but hide it from the list** — perfect for old or half-finished OpModes.

```
@TeleOp(name = "Old Drive - not used", group = "Archive")
@Disabled // <-- hides this OpMode from the Driver Station list
public class OldDrive extends LinearOpMode {
    @Override
    public void runOpMode() {
        // the code is still here, just not shown on the robot
    }
}
```

Buttons: act once, not every loop (and keep power in range)

The loop runs ~50 times a second, so `gamepad1.a` stays `true` the whole time a finger is on it. To **toggle** something (open/close a claw) on each press, catch the moment it *changes* — the "rising edge." And wrap stick values in `Range.clip` so a motor never gets more than full power.

```
import com.qualcomm.robotcore.util.Range;

boolean clawOpen = false;
boolean lastA    = false;    // what button A was on the PREVIOUS loop

while (opModeIsActive()) {
    // RISING EDGE: act once when A is first pressed (not every loop it's held)
    if (gamepad1.a && !lastA) {
        clawOpen = !clawOpen;    // flip it
    }
    lastA = gamepad1.a;

    // CLAMP so power can never leave the -1.0 .. 1.0 range
    double power = Range.clip(-gamepad1.left_stick_y, -1.0, 1.0);
    frontLeft.setPower(power);
}
```

Servos — set a position, not a power

A servo moves to an exact spot between `0.0` and `1.0` — great for claws, arms, and flippers.

```
Servo claw = hardwareMap.get(Servo.class, "claw");
claw.setPosition(0.0);    // position is 0.0 .. 1.0 (one end to the other)
claw.setPosition(1.0);
```

In TeleOp, never use sleep() — use a timer

`sleep()` freezes the **whole** robot, so the drivers lose control. In TeleOp, time things with `ElapsedTime` instead. (`sleep()` is fine in Autonomous, where nothing else is happening.)

```
ElapsedTime timer = new ElapsedTime();
timer.reset();
// ...later, inside the loop...
if (timer.seconds() > 2.0) {
    // happens 2 seconds later WITHOUT freezing the robot (unlike sleep)
}
```

Drive an exact distance with encoders

For precise autonomous, use the motors' built-in **encoders** (tick counters): reset them, set a target in ticks, then let the motor `RUN_TO_POSITION`.

```
frontLeft.setMode(DcMotor.RunMode.STOP_AND_RESET_ENCODER);
frontLeft.setTargetPosition(1120);    // ticks = how far to go
frontLeft.setMode(DcMotor.RunMode.RUN_TO_POSITION);
frontLeft.setPower(0.5);
while (opModeIsActive() && frontLeft.isBusy()) {
    telemetry.addData("pos", frontLeft.getCurrentPosition());
    telemetry.update();
}
```

As code grows, split it up

Don't copy the motor setup into every OpMode. Once you have 2-3 OpModes, make a **robot/hardware class** (a "subsystem") that configures the motors once and reuse it. Cleaner code = fewer bugs — and judges love it.

9 git — how a team codes together

git is version control: it saves snapshots ("commits") of your code so you can go back, and it lets the whole team work without clobbering each other's work. **GitHub** hosts the shared copy (the "repository"). The everyday loop:

```
# get the team's code onto your laptop (once)
git clone https://github.com/YourTeam/FtcRobotController.git

# make your own branch so you don't break the shared code
git checkout -b arcade-drive

# ...edit your OpMode in Android Studio...

git add .                # stage your changes
git commit -m "Add arcade drive OpMode" # save a snapshot with a message
git push origin arcade-drive # send it to GitHub for the team

git pull                # get teammates' latest changes
```

Team habit: commit small and often with clear messages, always work on your **own branch**, and `git pull` before you start so you have everyone's latest code. Never code for an hour without committing.

10 Good habits (that save matches)

- **Name hardware consistently** in the robot configuration AND in code — mismatched names are the #1 beginner bug.
- **Use telemetry** to see what the robot thinks is happening — print powers, sensor values, and state.
- **Test small** — get driving working before adding an arm; add one thing at a time.
- **Set `ZeroPowerBehavior` to BRAKE** for drive motors so the robot stops crisply.
- **Commit to git** after anything that works, so you can always get back to a working robot.

For coaches & mentors. Start new programmers on the SDK's built-in **sample OpModes** (in the `samples` folder — copy one into `TeamCode` and tweak it) rather than a blank file. Remember the **code is judged**: keep an engineering/coding notebook of what you tried and why. Put drive first, then one mechanism at a time. Reinforce **Gracious Professionalism** and get help from the official **FTC Docs** (ftc-docs.firstinspires.org), the **game manual**, and the FTC community forums. **Safety:** when testing wheels, put the robot up on a stand/blocks so it can't drive off the table, and keep hands clear of moving parts.

11 Glossary — words you'll hear

Term	What it means
------	---------------

OpMode	One program for the robot (TeleOp or Autonomous).
TeleOp	Driver-controlled mode — you drive with the gamepads.
Autonomous	The robot runs by itself, no driver.
LinearOpMode	The beginner OpMode style; code runs top-to-bottom in <code>runOpMode()</code> .
Robot Controller	The app on the Control Hub that runs your OpModes.
Driver Station	The phone/tablet with the gamepads and Init/Play/Stop.
Control Hub	The computer on the robot that runs the Robot Controller.
hardwareMap	How code grabs a motor/servo by its configured name.
DcMotor / Servo	Java objects for a drive motor / a servo.
gamepad1/2	The two controllers (buttons & sticks) on the Driver Station.
telemetry	Live text printed to the Driver Station — your debugger.
SDK	The FtcRobotController starter project you build on.
IDE	The app you write code in (Android Studio).
git / repo / commit / branch	Version control / the stored project / a saved snapshot / your own line of work.
encoder	A counter in a motor that measures how far it has turned (for precise auto).
@Disabled	Annotation that keeps an OpMode's code but hides it from the Driver Station list.
configuration	The Driver Station list that names each motor/servo and its port (names must match the code).
mecanum	A 4-wheel drivetrain that can drive, turn, AND strafe sideways.
robot-centric	"Forward" = the robot's front (simple; driver thinks from the robot's view).
field-centric	"Forward" = the same field direction regardless of robot heading (uses the IMU).
IMU	The orientation sensor in the Control Hub; reports the robot's heading (yaw).
Range.clip	Keeps a number inside a range (e.g. motor power within -1.0 .. 1.0).
ElapsedTime	A timer for doing things "later" without freezing the robot (used instead of sleep in TeleOp).
subsystem	A reusable class for one part of the robot (drive, arm...) so code isn't copied everywhere.